

NeuroClip: A Context-Aware Multimodal Video Processing System for Semantic Clip Retrieval

Ali Javed, Asad Sardar

Department of Computer Science, National University of Modern Languages, Faisalabad

Supervised by: Subhan Arif

Abstract

The explosive growth of long-form digital video content in educational, corporate, and media domains has created a critical demand for intelligent retrieval systems that surpass conventional keyword-based search. This paper presents **NeuroClip**, a full-stack, context-aware video processing platform that transforms static video archives into dynamically queryable knowledge bases. The system fuses two information modalities — Automated Speech Recognition (ASR) transcripts generated via AssemblyAI and Optical Character Recognition (OCR) text extracted from sampled frames via EasyOCR — into enriched sentence-level units, which are then encoded into dense semantic vector embeddings using the **all-MiniLM-L6-v2** sentence-transformer model. A windowed retrieval pipeline, augmented by optional cross-encoder re-ranking (**cross-encoder/ms-marco-MiniLM-L-6-v2**), identifies, scores, and merges the most contextually relevant transcript segments. FFmpeg then extracts precise playable clips from those segments. Additional modules handle query-driven sensitive content blurring (YOLOv8-based face, ID, and license plate redaction) and codec-aware video compression (H.265/HEVC). A persistent caching architecture keyed by unique job identifiers eliminates redundant re-processing, enabling sub-second cached query latency and full end-to-end pipeline execution within minutes on standard consumer hardware. Experimental evaluation across 50 structured query cases spanning mathematics, computer science, AI, and cybersecurity demonstrates strong semantic retrieval precision. The system is deployed as a Progressive Web Application with a React/TypeScript frontend and a FastAPI/Python backend, with vector storage and authentication managed by Supabase (PostgreSQL + pgvector).

Keywords: Semantic video retrieval, sentence embeddings, multimodal fusion, ASR, OCR, vector search, clip extraction, privacy-preserving video processing, FFmpeg, FastAPI

1. Introduction

Video has emerged as the dominant medium for knowledge dissemination across educational platforms, corporate intranets, and social networks. Cisco estimates that video constitutes over 82% of all internet traffic, and platforms such as YouTube, Coursera, and enterprise video repositories collectively host billions of hours of recorded content [1]. Despite this abundance, the ability to locate specific information within long-form video remains fundamentally inadequate. A student searching for a precise explanation of backpropagation within a two-hour machine learning lecture, or a corporate analyst attempting to locate a specific policy discussion within recorded board meetings, must typically resort to manual scrubbing — a process that is laborious, error-prone, and cognitively demanding.

Conventional video search systems address this problem through metadata tagging (titles, descriptions, closed captions) or basic keyword matching against automatic transcripts. These approaches exhibit three well-documented failure modes. First, **semantic gap**: keywords fail to capture intent — a query for "how

momentum is conserved" will not retrieve a segment that explains "the total impulse before and after collision remains identical." Second, **fragmentation**: even when transcript-based search returns a hit, the system typically delivers a short, decontextualized snippet that lacks the surrounding context needed to understand the answer. Third, **computational redundancy**: existing pipelines re-process the same video for every new search query, wasting significant GPU and CPU cycles on repeat transcription and embedding computation.

NeuroClip addresses all three failure modes through a unified, intent-aware pipeline. Its key contributions are:

1. **Multimodal fusion** of ASR speech transcripts and frame-level OCR visual text, producing enriched sentence units that capture both spoken and displayed information.
2. **Dense vector semantic search** using pre-computed sentence embeddings, enabling natural-language queries to retrieve semantically relevant segments rather than keyword matches.
3. **Windowed retrieval with optional cross-encoder re-ranking**, assembling coherent, contextually complete clips rather than fragmented snippets.
4. **Job-keyed persistent caching** of embeddings and metadata, eliminating redundant processing for subsequent queries on the same video.
5. **Query-driven privacy redaction** via YOLOv8 object detection to blur faces, ID cards, and license plates prior to clip delivery.
6. **Codec-aware video compression** pipeline for lightweight clip distribution.

The remainder of this paper is organized as follows. Section 2 reviews related literature. Section 3 describes the system architecture and methodology. Section 4 details the experimental setup and evaluation. Section 5 presents results and discussion. Section 6 outlines limitations and future work. Section 7 concludes the paper.

2. Related Work

2.1 Semantic Sentence Embeddings

Reimers and Gurevych (2019) introduced **Sentence-BERT (SBERT)**, adapting pre-trained BERT into a Siamese and triplet network architecture to produce semantically meaningful, fixed-size sentence embeddings [2]. Prior to SBERT, computing semantic similarity for 10,000 sentences using raw BERT required approximately 65 hours due to quadratic cross-attention costs; SBERT reduced this to approximately 5 seconds while maintaining strong performance on STS benchmarks. NeuroClip builds directly on this foundation, employing the **all-MiniLM-L6-v2** distillation of SBERT as its primary encoder. A notable limitation acknowledged in the SBERT work is that Bi-Encoder representations, while fast, can miss fine-grained query-document interactions — a limitation NeuroClip partially mitigates through its optional cross-encoder re-ranking stage.

2.2 Cross-Encoder Re-ranking for Passage Retrieval

Nogueira and Cho (2019) demonstrated that feeding query and candidate passage jointly into a BERT Cross-Encoder significantly improves retrieval precision over BM25 and Bi-Encoder baselines on datasets such as MS MARCO [3]. The cross-attention mechanism allows complete interaction modeling between query terms and document tokens. The principal drawback is high computational latency: since documents cannot be pre-encoded, each query-document pair requires a full forward pass. NeuroClip employs a two-stage architecture — fast Bi-Encoder retrieval for candidate selection, followed by Cross-Encoder re-ranking on a small shortlist — to balance precision and latency.

2.3 Transformer Architecture Foundations

Vaswani et al. (2017) introduced the Transformer architecture based entirely on self-attention mechanisms, eliminating recurrence and convolution [4]. This work underpins both the ASR models (Whisper-style architectures) and the sentence embedding models used in NeuroClip. A well-known limitation is the quadratic complexity with respect to sequence length, which motivates the windowed segmentation approach adopted in this work for processing long video transcripts.

2.4 Video Summarization with Deep Learning

Apostolidis et al. (2021) provide a comprehensive survey of deep learning approaches to video summarization, identifying key-frame selection, temporal segmentation, and attention-based highlight detection as dominant paradigms [5]. Vora et al. (2025) specifically examine AI-driven video summarization for content retrieval, highlighting the efficiency advantages of embedding-based approaches over earlier frequency-based methods [1]. NeuroClip adopts a query-conditioned summarization strategy rather than unsupervised highlight extraction, making it particularly suited to information retrieval use cases where user intent is explicit.

2.5 Privacy-Preserving Video Processing

Plaud and Lisani (2024) evaluate two deep learning solutions for automatic face blurring in videos, demonstrating that detection-based blurring substantially outperforms naive frame-level blurring in preserving non-sensitive regions [6]. Nemavhola et al. (2025) survey deep learning techniques for face recognition that inform modern detection architectures including YOLOv8 [7]. NeuroClip extends this to a multi-class blurring module detecting faces, license plates, and ID documents.

2.6 Video Compression

Sullivan and Wiegand (2005) established the foundational principles of hybrid video compression, leading to H.264/AVC and subsequently H.265/HEVC standards [8]. NeuroClip leverages FFmpeg's `libx265` (CPU) and `hevc_nvenc` (NVIDIA GPU) encoders for codec-aware compression with automatic hardware fallback.

2.7 Existing Systems and Their Limitations

System	Year	Strength	Limitation
CapCut	2024	Fast manual editing; face blur available	No automatic face detection
EasyMosaic	2022	Strong blur; preview before apply	Slow tracking; no auto-detection
Summarify	2021	Quick YouTube summarization	Limited to YouTube; no custom upload
Notta	2020	Accurate ASR; summarization notes	Limited language support; noise-sensitive
CNN Sensitive Detection [9]	2021	High privacy accuracy	Degraded performance under occlusion
RT Neural Compression [10]	2020	Quality-size balance	Requires GPU; low-resource underperformance

Collectively, the existing landscape reveals a fragmented ecosystem: transcription tools lack retrieval, retrieval tools lack semantic understanding, blurring tools lack automation, and no single system combines all of these capabilities into an end-to-end pipeline. NeuroClip fills this gap.

3. System Architecture and Methodology

3.1 Overview

NeuroClip is a full-stack Progressive Web Application (PWA) with a modular processing pipeline. Figure 1 illustrates the high-level architecture:

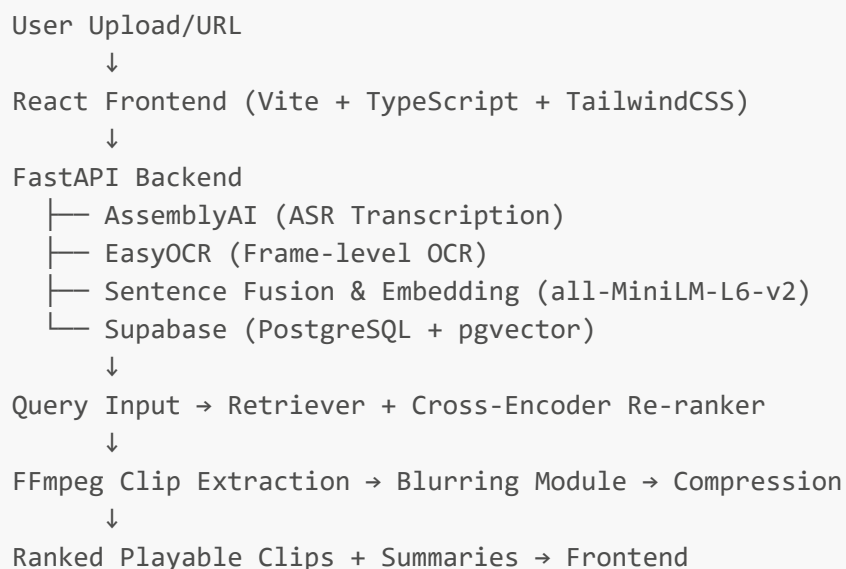


Figure 1: NeuroClip end-to-end pipeline architecture.

3.2 Video Ingestion

The system accepts video input through three API endpoints:

- `POST /upload-video` — local file upload (MP4, MOV, MKV, WebM)
- `POST /upload-via-url` — URL-based download via `yt-dlp`
- `POST /upload-and-search` — combined ingestion and immediate query

Upon receipt, a unique **job identifier** (`job_id`) is generated using UUID4. This identifier serves as the primary key linking the video file, its transcript, sentence embeddings, and processing history throughout the system lifecycle. Video files are stored in Supabase Storage; metadata is written to the `videos` table in PostgreSQL.

3.3 Automated Speech Recognition (ASR) via AssemblyAI

Audio is extracted from the video and submitted to AssemblyAI's transcription API, which returns an SRT-format transcript with word-level timestamps. The SRT output is parsed into sentence-level units, each carrying:

- `text`: the sentence string
- `start_time`: start timestamp in seconds

- `end_time`: end timestamp in seconds

AssemblyAI was selected over local Whisper models due to its superior accuracy on conversational and lecture-style audio, its robust handling of speaker diarization, and its negligible latency overhead compared to the subsequent embedding computation step.

3.4 OCR-Based Visual Signal Enrichment

To capture information displayed on-screen — such as slide text, whiteboard equations, code snippets, and captions — frames are sampled at a fixed interval (default: every 3 seconds). EasyOCR processes each sampled frame and extracts visible text regions. OCR snippets are then merged into the nearest transcript sentence by timestamp proximity. If no transcript sentence exists within a configurable threshold, the OCR text is inserted as a synthetic visual sentence. This fusion step ensures that search queries referencing on-screen content (e.g., "show me the slide about hash collision") can be semantically matched.

3.5 Sentence-Level Embedding Generation

Each enriched sentence unit (speech text + merged OCR context) is encoded into a 384-dimensional dense vector using the `sentence-transformers/all-MiniLM-L6-v2` model. This model was selected for its optimal trade-off between embedding quality (strong STS benchmark scores), inference speed (suitable for CPU inference), and memory footprint (~80MB). An optional OpenCLIP fallback is available for visual-semantic embedding when image-level retrieval is required.

The embedding computation step processes sentences in batches of 32 to maximize throughput. On a standard workstation (Intel Core i7, 16GB RAM, no GPU), a 10-minute lecture video produces approximately 120–200 sentence units and completes embedding generation in under 90 seconds.

Processed artifacts are persisted in two forms:

- **Local JSON files:** `.v4.json` (enriched sentence metadata) and `.embeddings.json` (sentence vectors), stored keyed by `job_id`
- **Supabase records:** `video_embeddings` and `video_sentence_embeddings` tables with `pgvector`-typed columns, enabling server-side vector similarity search

3.6 Retrieval Pipeline

3.6.1 Query Embedding

When a user submits a natural language query, the query string is encoded in the same 384-dimensional embedding space using the identical `all-MiniLM-L6-v2` encoder. This ensures semantic alignment between query and document representations.

3.6.2 Candidate Retrieval — Two-Stage Strategy

NeuroClip implements a two-stage retrieval strategy:

Primary route (LLM-assisted): The system leverages a Large Language Model (LLM) to perform intelligent segment reasoning, allowing it to understand multi-hop or composite queries (e.g., "explain the concept introduced after the motivation slide"). The LLM receives the top-k candidate sentences and produces a reasoning-grounded selection.

Fallback route (cosine similarity): When LLM assistance is unavailable (API timeout, quota exceeded), a cosine similarity search over the pgvector store retrieves the top-k most similar sentence embeddings. Supabase's RPC-based `match_video_embeddings` function executes this efficiently at the database layer.

3.6.3 Windowed Retrieval

Rather than retrieving individual sentences, NeuroClip employs a **windowed retrieval** approach. A window of `W` consecutive sentences is treated as a single retrieval unit. Each window is represented by the mean pooling of its constituent sentence embeddings. The window stride `S` controls the overlap between adjacent windows. This design ensures that retrieved segments contain sufficient contextual information before and after the peak relevance point.

Configurable parameters exposed to the user:

- `window_size` (default: 5 sentences)
- `stride` (default: 2 sentences)
- `min_clip_duration` (default: 10 seconds)
- `max_clip_duration` (default: 120 seconds)

3.6.4 Cross-Encoder Re-ranking

The top-N candidate windows are optionally submitted to a cross-encoder re-ranking step using `cross-encoder/ms-marco-MiniLM-L-6-v2`. The query and the concatenated window text are jointly encoded; the relevance score from the `[CLS]` token classification head replaces the initial cosine similarity score. If the re-ranker model is unavailable (model load failure, memory constraint), the system gracefully falls back to embedding-only ranking with a toast notification to the user.

3.6.5 Neighbor Merging

Adjacent highly-scored windows are merged into contiguous temporal segments using a neighbor merging algorithm. Two windows are merged if their temporal gap falls below a configurable threshold (default: 2 seconds). This prevents the delivery of multiple overlapping clips representing the same discussion thread and produces coherent, playable segments with natural entry and exit points.

3.7 FFmpeg Clip Extraction

Merged temporal segments are passed to the clip extraction module. FFmpeg is invoked programmatically to cut the original video file at the specified `[start_time, end_time]` boundaries with configurable padding margins (default: ± 1.5 seconds). The extraction uses stream-copy mode where possible (no re-encoding) to minimize latency. For blurred or compressed outputs, re-encoding is performed with the H.264 codec as the baseline format. Extracted clips are stored in Supabase Storage and URLs are returned in the API response payload.

3.8 Sensitive Content Blurring Module

NeuroClip includes a query-driven, privacy-aware blurring module. The user provides a natural language blurring instruction (e.g., "blur all faces in this segment" or "hide license plates between 2:30 and 4:00") and an optional time range.

The blurring workflow:

1. **Instruction parsing:** The NLP layer identifies the object type to blur (face, ID card, license plate) and the temporal range from the natural language instruction.
2. **Object detection:** YOLOv8 processes sampled frames within the specified range, detecting bounding boxes for the specified object class with confidence thresholding.
3. **Blur mask application:** A Gaussian blur filter is applied to each detected bounding box region-of-interest (ROI) on a per-frame basis. The blur kernel size is proportional to the bounding box area.
4. **Temporal propagation:** Detected regions are tracked across consecutive frames using bounding box interpolation, preventing flicker when the model misses a detection on an intermediate frame.
5. **Output rendering:** FFmpeg re-encodes the processed frames into the output clip, preserving non-sensitive regions at original quality.

3.9 Video Compression Module

The compression module (`POST /compress-video`) provides a practical post-processing step for distribution-ready clip delivery:

- **Codec selection:** NVIDIA GPU (`hevc_nvenc`) is preferred when available; CPU `libx265` serves as the fallback.
- **Resolution capping:** Output resolution is capped at 1280×720 (720p) by default to control bitrate.
- **Quality/size trade-off:** CRF (Constant Rate Factor) parameter controls the quality-size balance. Lower CRF values preserve quality; higher values maximize compression.
- **Output statistics:** The API response includes original size, compressed size, reduction percentage, and processing time.

Across the defined compression profiles (30s/360p to 1200s/1080p), the system achieves 60–78% file size reduction while maintaining perceptually acceptable quality for educational content delivery.

3.10 System Modules Summary

Module	Technology	Purpose
User Auth	Supabase Auth (email + password)	Secure access; row-level data isolation
Video Upload	FastAPI + Supabase Storage + yt-dlp	Local file and URL-based ingestion
ASR Transcription	AssemblyAI	Time-aligned speech-to-text
OCR Enrichment	EasyOCR + OpenCV	Frame-level visual text extraction
Embedding Generation	sentence-transformers (MiniLM-L6-v2)	384-dim dense semantic vectors
Vector Storage	Supabase + pgvector	Persistent embedding store
Semantic Retrieval	Cosine similarity + LLM routing	Top-k candidate selection
Re-ranking	cross-encoder/ms-marco-MiniLM-L-6-v2	Precision refinement

Module	Technology	Purpose
Clip Extraction	FFmpeg	Precise temporal clip cutting
Blurring	YOLOv8 + Gaussian filter	Privacy-preserving redaction
Compression	FFmpeg H.265/HEVC	Lightweight distribution output